



深圳職業技術大學

SHENZHEN POLYTECHNIC UNIVERSITY

数据结构课程

迷宫求解的三种方法项目

实验实训报告

一、实验实训项目名称

迷宫求解的三种方法

二、项目内容与要求

1、实验目的

掌握递归、栈（DFS）、队列（BFS）三种方法求解迷宫问题的核心原理与实现 逻辑。 对比三种方法的路径特性（是否最短）、时间效率、空间占用及适用场景。 理解深度优先搜索（DFS）与广度优先搜索（BFS）的本质差异。 熟练运用数据结构（栈、队列）模拟搜索过程，解决实际路径规划问题。

实验内容

迷宫规格：5×5 网络迷宫（可扩展至 10×10），定义如下：

0：可通行区域

1：障碍物 起点

S：坐标（0,0） 终点 E：坐标（4,4）

迷宫矩阵示例：

```
int maze[5][5] = { {0, 1, 0, 0, 0}, {0, 1, 0, 1, 0}, {0, 0, 0, 1, 0}, {0, 1, 1, 1, 0}, {0, 0, 0, 0, 0} };
```

1. 核心基础 迷宫问题本质是 “图的路径搜索”，网格中每个可通行位置为 “顶点”，相邻可通行位置（上、下、左、右）为 “边”，求解目标是找到从起点到终点的有效路径。
2. 三种方法的核心逻辑（1）递归法（DFS） 核心策略：深度优先，“一条路走到黑”，走不通则回溯。实现逻辑： 终止条件：当前位置为终点（返回成功）或越界 / 障碍物 / 已访问（返回失败）。 递归步骤：标记当前位置为已访问，依次探索四个方向，递归调用自身；若某方 向返回成功，则沿该路径回溯返回。（2）栈法（DFS） 核心策略：手动模拟递归的调用栈，实现深度优先搜索。实现逻辑： 栈存储待探索状态（坐标、已探索方向）。初始状态（起点、方向 0）入栈，标记起点为已访问。 栈非空时，弹出栈顶元素：若为终点则成功；否则依次探索未尝试方向，可行状态入栈（标记已访问）。（3）队列法（BFS） 核心策略：广度优先，“层层推进”，首次到达终点即为最短路径。实现逻辑： 队列存储待探索状态（坐标、前驱节点索引，用于重建路径）。 起点入队，标记为已访问。 队列非空时，出队当前元素，探索所有相邻可行位置，入队并记录前驱；若某位 置为终点，则通

过前驱回溯路径。

三、报告内容

1、方法、步骤：

(1). 实验准备

定义迷宫矩阵 `maze[N][N]`，其中 0 表示通路，1 表示障碍。

定义起点 `sx, sy`，终点 `ex, ey`。

定义方向数组：

```
int dx[] = {-1, 1, 0, 0};
```

```
int dy[] = {0, 0, -1, 1};
```

分别表示上、下、左、右四个方向。

定义 `valid(int x, int y)` 判断坐标是否越界。

路径使用 `vector<pair<int,int>>` 存储，便于统一输出。

(2). 递归法实现步骤

1. 编写递归函数 `mazeRecursion(x, y, path, visited)`。
2. 先判断当前位置是否越界、是否为障碍、是否已访问，若不满足则返回 `false`。
3. 若当前位置可走，则标记已访问，并加入路径。
4. 若当前位置为终点，则返回 `true`。
5. 按上、下、左、右四个方向递归搜索，若某一方向到达终点则返回 `true`。
6. 若四个方向都失败，则回溯，删除当前路径结点，返回 `false`。

(3). 栈法实现步骤

1. 定义栈结点 `State`，包含坐标 `x、y` 和方向标记 `dir`。
2. 起点入栈并标记访问，同时加入路径。
3. 循环处理栈顶元素：若当前为终点则返回成功。
4. 若 `dir` 等于 4，说明四个方向都已尝试，执行回溯：出栈、删除路径结点，并让上一个结点继续尝试下一个方向。
5. 否则根据 `dir` 计算下一位置，若不可走则 `dir` 加 1。
6. 若下一位置可走，则压栈、标记访问并加入路径。
7. 栈法通过 `dir` 记录搜索进度，从而模拟递归过程。

(4). 队列法实现步骤

1. 定义队列 `q`、访问标记数组 `visited` 和前驱数组 `parent`。
2. 起点入队并标记访问。
3. 循环出队当前结点，按上、下、左、右四个方向扩展相邻位置。
4. 若新位置可走且未访问，则入队，并记录其前驱。
5. 若终点可达，则从终点根据 `parent` 逐步回溯到起点，得到逆序路径。
6. 将路径反转后输出，得到从起点到终点的正序结果。

(5). 主函数实现步骤

1. 定义 `recursionPath`、`dfsPath`、`bfsPath` 三个路径容器。
2. 分别调用递归法、栈法和队列法求解迷宫。
3. 调用 `printPath()` 输出三种方法的路径和步数。
4. 对比三种方法的结果，分析其搜索特点。

2、实验步骤 (源代码和程序运行截图)

初始化

```
4  const int N = 5;
5  int maze[N][N] = {
6      {0, 1, 0, 0, 0},
7      {0, 1, 0, 1, 0},
8      {0, 0, 0, 1, 0},
9      {0, 1, 1, 1, 0},
10     {0, 0, 0, 0, 0}};
11  int sx = 0, sy = 0;
12  int ex = 4, ey = 4;
13  int dx[] = {-1, 1, 0, 0};
14  int dy[] = {0, 0, -1, 1};
15  int visited[N][N] = {0};
16  vector<pair<int, int>> recursionPath;
17  vector<pair<int, int>> dfsPath;
18  vector<pair<int, int>> bfsPath;
```

递归法代码

```
bool valid(int x, int y)
{
    return x >= 0 && x < N && y >= 0 && y < N;
}

bool mazeRecursion(int x, int y, vector<pair<int, int>> &path, int visited[][N])
{
    if (!valid(x, y) || maze[x][y] == 1 || visited[x][y])
        return false;

    visited[x][y] = 1;
    path.push_back({x, y});

    if (x == ex && y == ey)
        return true;

    for (int i = 0; i < 4; i++)
    {
        if (mazeRecursion(x + dx[i], y + dy[i], path, visited))
            return true;
    }

    visited[x][y] = 0;
    path.pop_back();
    return false;
}
```

DFS（栈）法代码

```

bool mazeDfs(vector<pair<int, int>> &path)
{
    struct State
    {
        int x;
        int y;
        int dir; // 保存方向信息
    };

    int visited[N][N] = {0};
    stack<State> st;
    path.clear();

    if (!valid(sx, sy) || !valid(ex, ey) || maze[sx][sy] == 1 || maze[ex][ey] == 1)
        return false;

    st.push({sx, sy, 0});
    visited[sx][sy] = 1;
    path.push_back({sx, sy});

    while (!st.empty())
    {
        State &cur = st.top();
        if (cur.x == ex && cur.y == ey)
            return true;

        if (cur.dir == 4)
        {
            visited[cur.x][cur.y] = 0;
            st.pop();
            path.pop_back();
            if (!st.empty())
                st.top().dir++;
            continue;
        }

        int nx = cur.x + dx[cur.dir];
        int ny = cur.y + dy[cur.dir];
        if (!valid(nx, ny) || maze[nx][ny] == 1 || visited[nx][ny])
        {
            cur.dir++;
            continue;
        }

        visited[nx][ny] = 1;
        st.push({nx, ny, 0});
        path.push_back({nx, ny});
    }

    return false;
}

```

BFS（队列）法代码

```

bool mazeBfs(vector<pair<int, int>> &path)
{
    int visited[N][N] = {0};
    pair<int, int> parent[N][N];
    queue<pair<int, int>> q;
    path.clear();

    if (!valid(sx, sy) || !valid(ex, ey) || maze[sx][sy] == 1 || maze[ex][ey] == 1)
        return false;

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            parent[i][j] = {-1, -1};
        }
    }

    q.push({sx, sy});
    visited[sx][sy] = 1;

    while (!q.empty())
    {
        pair<int, int> cur = q.front();
        q.pop();

        if (cur.first == ex && cur.second == ey)
            break;

        for (int i = 0; i < 4; i++)
        {
            int nx = cur.first + dx[i];
            int ny = cur.second + dy[i];
            if (!valid(nx, ny) || maze[nx][ny] == 1 || visited[nx][ny])
                continue;

            visited[nx][ny] = 1;
            parent[nx][ny] = cur;
            q.push({nx, ny});
        }
    }

    if (!visited[ex][ey])
        return false;

    for (pair<int, int> cur = {ex, ey}; cur.first != -1; cur = parent[cur.first][cur.second])
    {
        path.push_back(cur);
    }
    reverse(path.begin(), path.end());
    return true;
}

```

统一的打印函数

```

void printPath(const char *title, const vector<pair<int, int>> &path)
{
    printf("%s", title);
    printf("(%d,%d)", path[0].first, path[0].second);
    for (size_t i = 1; i < path.size(); i++)
    {
        printf("->(%d,%d)", path[i].first, path[i].second);
    }
    printf("  步数:%d\n", (int)path.size() - 1);
}

```

主程序

```
int main()
{
    mazeRecursion(sx, sy, recursionPath, visited);
    mazeDfs(dfsPath);
    mazeBfs(bfsPath);

    printPath("递归: ", recursionPath);
    printPath("DFS:", dfsPath);
    printPath("BFS:", bfsPath);

    return 0;
}
```

实验结果:

```
[Running] cd "d:\study\DataStructure\lab04\" && g++ lab04.cpp -o lab04 && "d:
递归: (0,0)->(1,0)->(2,0)->(3,0)->(4,0)->(4,1)->(4,2)->(4,3)->(4,4) 步数:8
DFS:(0,0)->(1,0)->(2,0)->(3,0)->(4,0)->(4,1)->(4,2)->(4,3)->(4,4) 步数:8
BFS:(0,0)->(1,0)->(2,0)->(3,0)->(4,0)->(4,1)->(4,2)->(4,3)->(4,4) 步数:8
```

3、实验结论及思考:

通过本次实验，分别使用递归、栈和队列三种方法实现了迷宫求解，并成功输出了从起点到终点的路径。递归法和栈法本质上都属于深度优先搜索，因此在搜索顺序一致时得到的路径相同；队列法属于广度优先搜索，能够保证找到最短路径。

通过对三种方法的比较，可以看出递归法实现最直观，但依赖系统调用栈；栈法能够用非递归方式模拟深度优先搜索过程；队列法层次清晰，更适合求最短路径